

FTC Rookie Java Bootcamp

Prerequisites:

- [Blocks Programming Tutorial — FIRST Tech Challenge Docs 0.3 documentation](#)
- [OnBot Java Programming Tutorial — FIRST Tech Challenge Docs 0.3 documentation](#)
- **Even if you are experienced review -**
<https://github.com/first-tech-challenge/ftcrobotcontroller>

Technical backbone: [Learn Java for FTC by Alan Smith](#)

Repository: <https://github.com/isardam/rookieftcjava>

This bootcamp uses Java as a robotics tool rather than teaching it as a traditional programming class. Each session ends with a working project, building from basic Java concepts toward an FTC OpMode.

Recommended Learning Environment

Tool	Use	Sessions
CodeHS	https://codehs.com/explore/sandbox/java - without login Create login if you want to save your work	1–6
FTC OnBot Java	First hands-on FTC Java environment	7–8
Android Studio	Professional FTC development environment	After Session 8
Programiz	Optional Java reference and runnable examples	As needed

Important: CodeHS is not an FTC simulator. It is used to learn Java and programming logic without the overhead of robot configuration. Students transition to actual FTC Java after the foundational concepts.

Bootcamp Goal

By the end of eight sessions, every student should be able to read, explain, modify, and begin writing an FTC Java OpMode using variables, decisions, loops, methods, classes/objects, gamepad input, and telemetry.

Project Roadmap

Session	Java Focus	Robotics Project	Finished Product
1	Java basics	Mission Control	MissionControl.java
2	Variables & data types	Robot Dashboard	RobotDashboard.java
3	Math & decisions	Decision Engine	RobotDecisionEngine.java
4	Loops	Search Robot	SearchBot.java
5	Methods & parameters	Command Library	RobotCommandLibrary.java
6	Classes & objects	Virtual Robot	VirtualRobot.java
7	FTC Java, gamepad & telemetry	Driver Dashboard	IronAngelTeleOp.java
8	Integration	Autonomous Mission	Mini FTC program

Session 1: Meet Your Robot: Java Controls a Mission

Java Concepts

- Java basics
- Program structure
- Classes and methods
- Comments
- Statements and semicolons
- Basic output

Project

Students create a Mission Control program that prints a robot's mission sequence.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Introduce the question: How does a robot know what to do?
- Live-code a simple Java program.
- Students build MissionControl.java.
- Add a debugging challenge.
- Students modify the mission and demonstrate it.

Deliverable

MissionControl.java

Session 2: Robot Brain: Variables & Data

Java Concepts

- int
- double
- boolean
- String
- Assignment
- Naming conventions

Project

Students build a Robot Dashboard that reports battery, motor power, target status, and robot name.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Introduce data as the robot's information.
- Create variables representing robot state.
- Print a formatted dashboard.
- Give teams different scenarios and have them change the values.
- Discuss examples such as distance, encoder count, and sensor state.

Deliverable

RobotDashboard.java

Session 3: Make the Robot Think

Java Concepts

- Arithmetic
- Comparisons
- if
- else
- else if
- Boolean logic

Project

Students build a Robot Decision Engine that chooses SEARCH, APPROACH TARGET, or RETURN HOME.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Build a decision tree before coding.
- Translate pseudocode into Java.
- Test several robot scenarios.
- Introduce edge cases.
- Have teams explain why their robot made each decision.

Deliverable

RobotDecisionEngine.java

Session 4: Loops: Teach the Robot to Repeat

Java Concepts

- for loops
- while loops
- Counters
- Repeated actions
- FTC loop behavior

Project

Students build a Grid Search Robot that systematically searches a field.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Compare repeated code with a loop.
- Build a 4×4 search pattern.
- Challenge teams to find a target efficiently.
- Introduce the difference between a normal Java loop and FTC's repeated loop() calls.
- Finish with a timed team challenge.

Deliverable

SearchBot.java

Session 5 — Build Your Own Robot Functions

Java Concepts

- Methods
- Parameters
- Return values
- Class members
- Reusable code

Project

Students create a Robot Command Library with reusable actions such as moveForward, turnLeft, search, deliver, and returnHome.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Demonstrate why giant programs are difficult to maintain.
- Create methods with parameters.
- Build a shared command library.
- Have one student use another student's methods.
- Discuss readable, reusable code.

Deliverable

RobotCommandLibrary.java

Session 6 — Object-Oriented Robot

Java Concepts

- Classes
- Objects
- Constructors
- this
- Basic encapsulation

Project

Students create a Virtual Robot object with position, heading, battery, and target state.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Create a Robot class.
- Instantiate a robot object.
- Give the robot actions such as move(), turn(), and scan().
- Display the robot's changing state.
- Connect the concept to FTC objects such as motors, servos, sensors, gamepads, and telemetry.

Deliverable

VirtualRobot.java

Session 7 — Welcome to FTC Java

Java Concepts

- OpMode structure
- gamepad1
- telemetry
- init()
- loop()
- FTC SDK concepts

Project

Students create a Gamepad Dashboard that displays joystick values, buttons, and robot modes.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Move into the actual FTC Java environment.
- Create a basic TeleOp OpMode.
- Read joystick and button values.
- Display information through telemetry.
- Implement normal/turbo modes.
- Connect previous Java concepts to FTC programming.

Deliverable

IronAngelTeleOp.java

Session 8 — FTC Mini Software Challenge

Java Concepts

- Integration of all prior concepts
- Debugging
- Autonomous logic
- FTC OpMode design

Project

Teams program an Autonomous Mission: leave base, find a target, navigate around an obstacle, deliver the object, and return home.

2-Hour Session Plan

- 0–10 min: Hook / robotics problem
- 10–25 min: Concept introduction
- 25–45 min: Instructor live coding
- 45–75 min: Team project build
- 75–90 min: Debugging challenge
- 90–110 min: Extension / team challenge
- 110–120 min: Demo and reflection

Activities

- Give teams a field diagram and mission requirements.
- Require variables, decisions, loops, methods, classes/objects, telemetry, and FTC OpMode structure.
- Introduce a surprise change in the field or target location.
- Run a team demo/competition.
- Have students explain their design and debugging decisions.

Deliverable

Mini FTC program

The Programming Cycle

- THINK — What does the robot need to do?
- CODE — Translate the plan into Java.
- TEST — Run the program.
- BREAK — Intentionally test an edge case or introduce a bug.
- FIX — Debug the problem.
- DEMO — Show something working before leaving.

Java-to-FTC Mental Model

Java Concept	FTC Connection
Variable	Sensor, gamepad, or robot-state data
if/else	Robot decision
Loop	Repeated robot control
Method	Reusable robot action
Class	Robot component or software design
Object	Motor, sensor, servo, gamepad, etc.
boolean	Button or sensor state
Telemetry	Information the robot sends to the driver

Must learn:

- Variables and data types
- Math
- Boolean logic
- if/else
- Loops
- Methods and parameters
- Classes and objects
- FTC OpMode structure
- Gamepad input
- Telemetry

Advanced concepts for later:

- Advanced inheritance
- Interfaces in depth
- Generics in depth
- Collections in depth
- Exceptions in depth
- Threads
- Advanced computer vision
- Sophisticated software architecture

Coach Notes

The bootcamp is not a copy-and-paste exercise. Students should write pseudocode first, predict what the program will do, run it, break it, and explain the fix.

The biggest goal is not syntax memorization. It is developing the habit of thinking like a robotics engineer: break a problem into smaller behaviors, represent the robot's state, make decisions, repeat actions, build reusable components, test assumptions, and debug systematically.

After Session 8

Transition to Android Studio and the FTC SDK once the students have a basic Java mental model. At that point, the setup complexity becomes useful rather than overwhelming. Begin connecting the same software concepts to real motors, servos, sensors, encoders, IMU data, and autonomous routines.